

Pruebas de Software - INF331

PRESENTACIÓN 2

INTEGRACIÓN

Grupo 4

Iñaki Oyarzun M. - Luis Gonzalez A. - Fernando Delgado

Profesor: Oscar Reyes



INTRODUCCIÓN

App Web

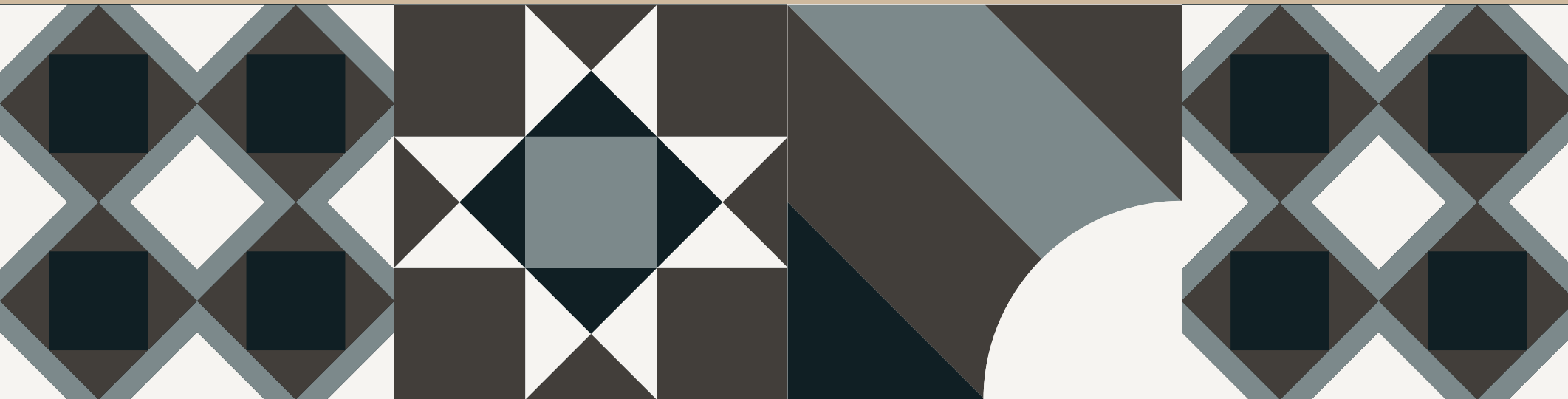
Uso de Django

Selenium

Jenkins

Slack

Jira



SELENIUM

Ecosistema open source para la ejecución de pruebas en navegadores.



DJANGO

Framework para el desarrollo web,
basado en python, Open Source



Y OTRAS MÁS



Jenkins



REQUERIMIENTOS



Manejo de Recetas



Crud para recetas



posibilidad de ser
testeada

SUPUESTOS



-Se asume campos de imagen será exclusivo para imágenes

-Hardware apto para usar como server web y realizar pruebas

REQUERIMIENTOS DEL NEGOCIO

Solo los atributos válidos para una
receta válida



REQUERIMIENTOS FUNCIONALES

*Usuario debe poder
agregar receta*

Todas las recetas
agregadas deben ser
visibles desde menú
principal

*Usuario debe poder
eliminar una receta*

*Usuario debe poder
editar datos de receta*

*Usuario debe poder
visualizar receta*

REQUERIMIENTOS NO FUNCIONALES

App debe ser compatible con navegador Chrome

App debe ser compatible con testing de Selenium

Usuario debe poder agregar una receta sin ayuda externa

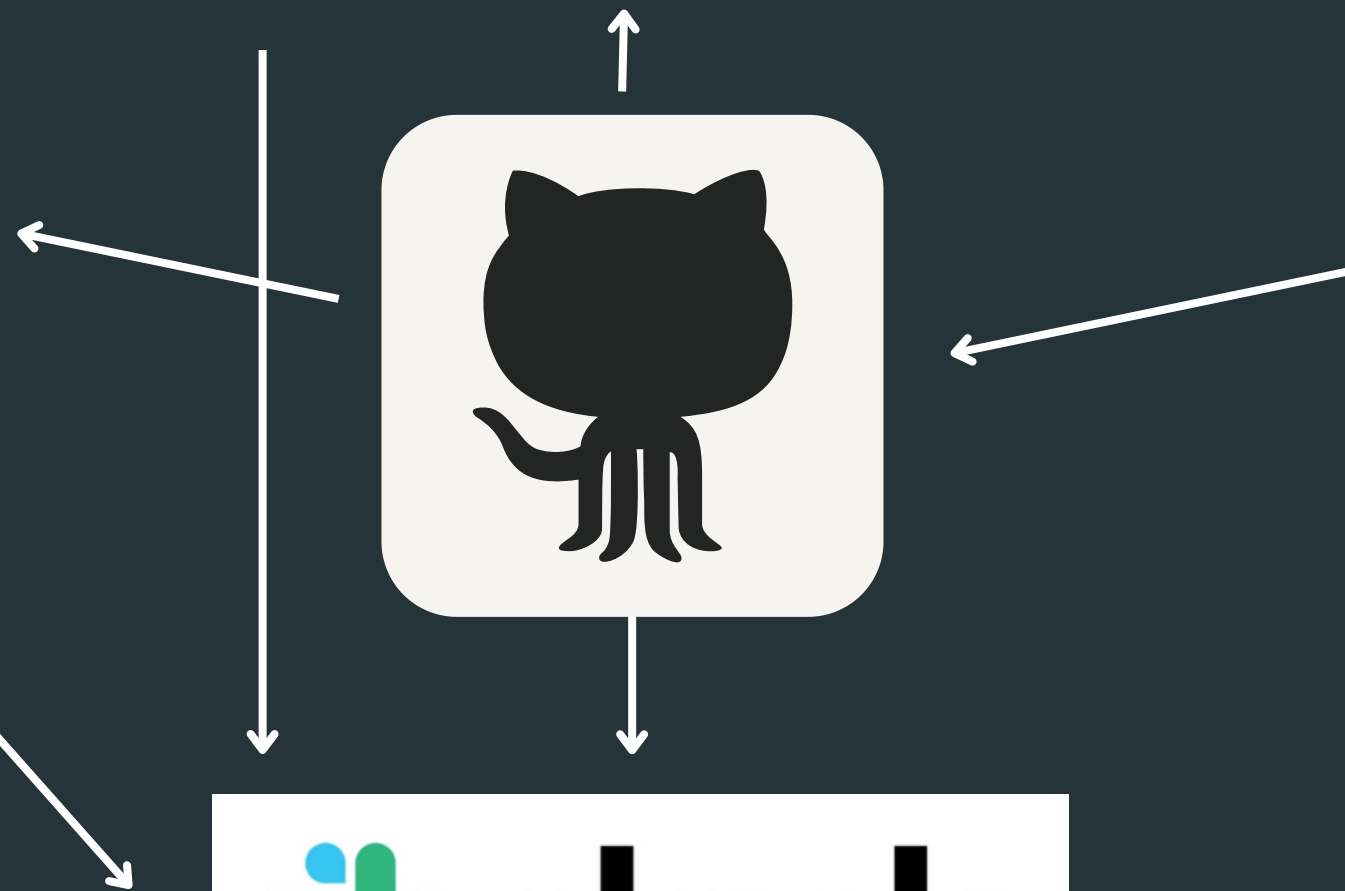
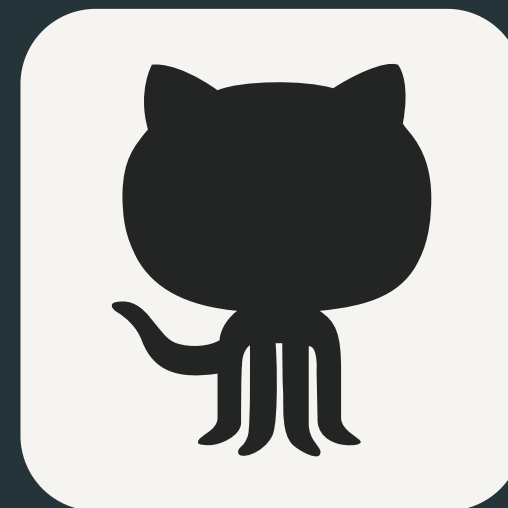
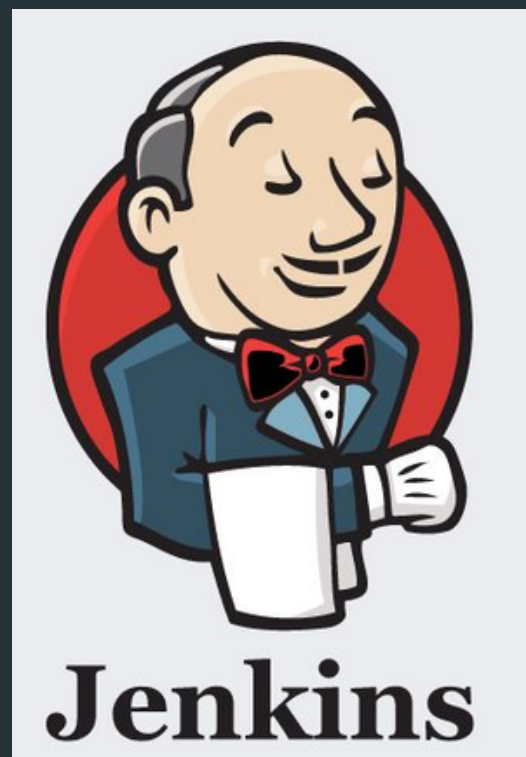
Sistema posee suite de testing integrada para facilitar la ejecución de pruebas

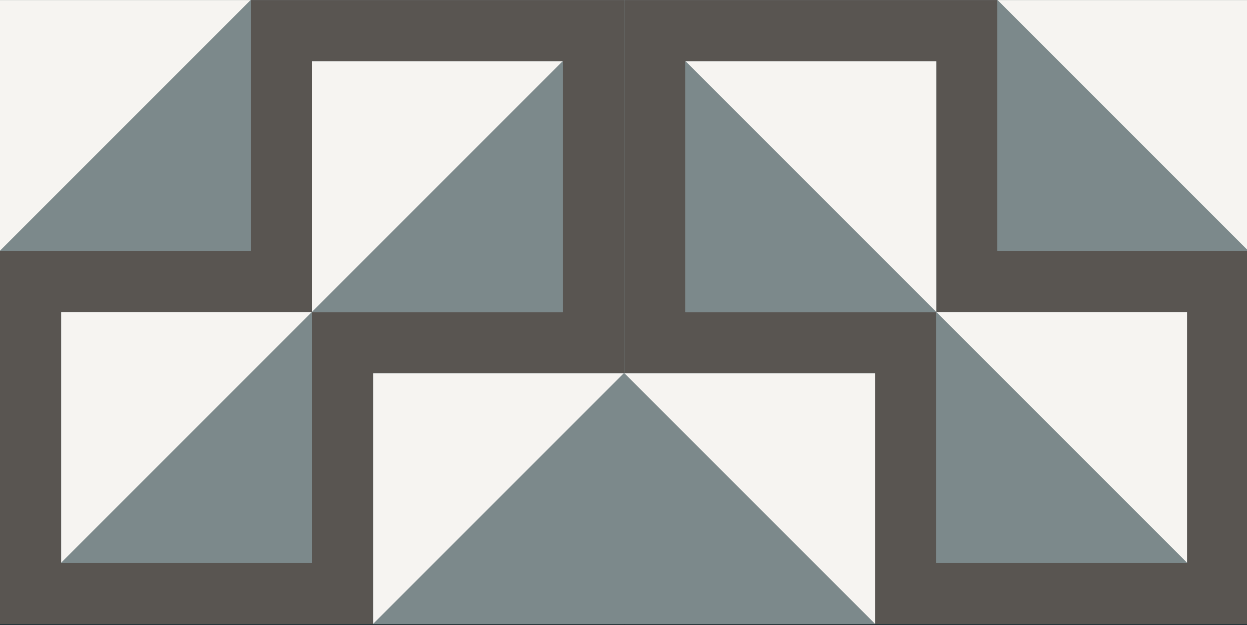
PRIORIDAD REQUERIMIENTOS

- *(M)ust: Obligatorio*
- *(S)hould: Alta prioridad*
- *(C)ould: Preferible pero no necesario*
- (W)ould: Puede ser pospuesto para ejecución a futuro

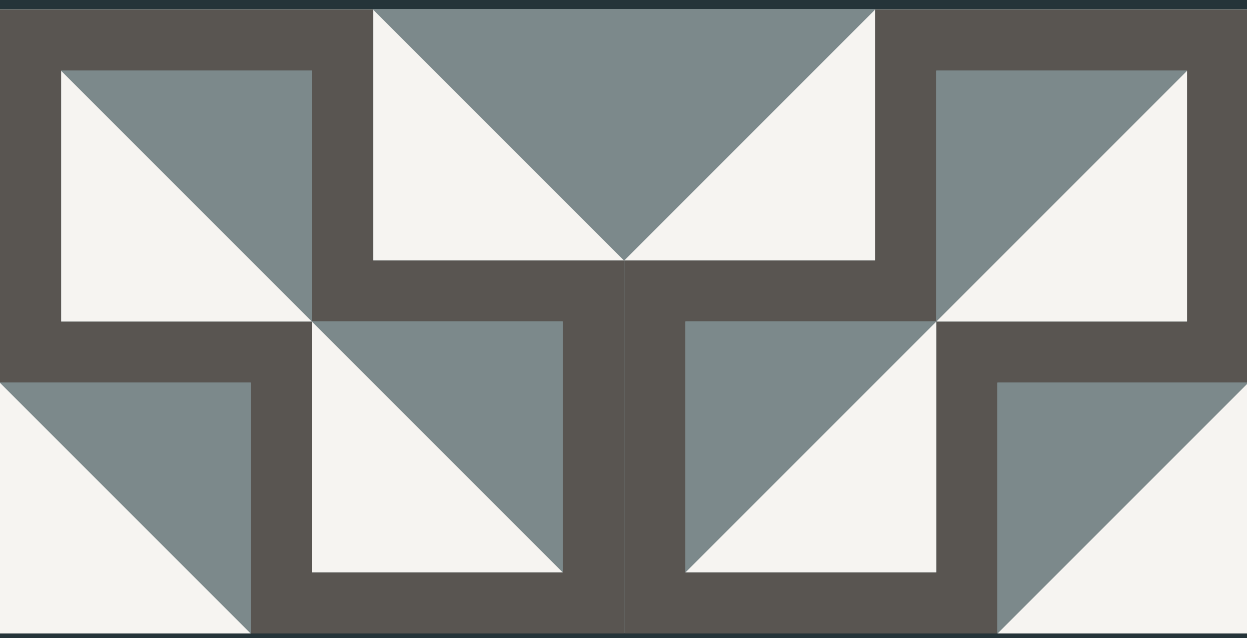
Requerimiento	Prioridad
El usuario debe poder agregar una receta	M
Todas las recetas agregadas deben ser visibles desde el menú principal	M
El usuario debe poder eliminar una receta	M
El usuario debe poder editar los datos una receta	M
El usuario debe poder visualizar los detalles de una receta	M
La aplicación debe ser compatible con el navegador Chrome	M
La aplicación debe ser compatible con la librería de pruebas Selenium	M
Deben poder realizarse exitosamente pruebas automatizadas con Jenkins/Selenium	M
Un usuario sin experiencia previa debe poder agregar una receta sin necesidad de ayuda externa	S
El sistema cuenta con testabilidad tal que se pueden realizar pruebas unitarias automatizadas para cada función	M

DEPENDENCIAS ENTRE TECNOLOGÍAS





PRUEBAS AUTOMATIZADAS



5 escenarios

Agregar una receta con el formato incorrecto – Ingredientes

Agregar una receta con el formato incorrecto – Pasos

Agregar una receta con falta de información – Imagen

Ejecución de CRUD – Unit Testing

```
def test_VerReceta(self):
    wait = WebDriverWait(self.driver, 15)
    ## Creacion de la sopaipilla
    self.driver.get(self.live_server_url)
    WebDriverWait(self.driver,10).until(EC.presence_of_element_located((By.ID,'explanation')))
    poll = self.driver.find_element(By.ID,'explanation')
    assert poll.text == "Ver recetas"
    button1 = self.driver.find_element(By.ID,"anadirReceta")
    ActionChains(self.driver).move_to_element(poll).click(button1).perform()
    menu = self.driver.find_element(By.ID,'MENU_ADDRECETA')
    assert menu.text == "Añadir receta"
    nombre = self.driver.find_element(By.NAME, "nombreReceta")
    nombre.send_keys("Sopaipilla")
    ingredientes = self.driver.find_element(By.NAME, "ingredientes")
    ingredientes.send_keys("Harina\nHuevos\nZapallo")
    pasos = self.driver.find_element(By.NAME, "pasos")
    pasos.send_keys("Paso1\nPaso2\nPaso3\n")
    autor = self.driver.find_element(By.NAME, "autor")
    autor.send_keys("ABCDEFGHJKLMNOPQRSTUVWXYZABCD1234")
    imagen = self.driver.find_element(By.NAME, "imagenReceta")
    imagen.send_keys(get_local_path()+"\sopaipilla.jpg")
    buttonadd = self.driver.find_element(By.ID, "BOTON_ADD_RECETA")
    element = wait.until(EC.invisibility_of_element_located((By.ID,"failed-alert")))
    ActionChains(self.driver).click(buttonadd).perform()
    wait.until(EC.visibility_of_element_located((By.ID,"success-alert")))
    wait.until(EC.invisibility_of_element_located((By.ID,"success-alert")))

    ## Acceso a la información de la sopaipilla
    ActionChains(self.driver).click(self.driver.find_element(By.ID, "verRecetas")).perform()
    wait.until(EC.presence_of_element_located((By.ID,'explanation')))
    poll = self.driver.find_element(By.ID,'explanation')
    assert poll.text == "Ver recetas"
    wait.until(EC.presence_of_element_located((By.NAME,'verReceta')))
    texto = self.driver.find_element(By.NAME,"verReceta")
    assert texto.text == "Ver Receta"
    ActionChains(self.driver).click(texto).perform()
    texto2 = self.driver.find_element(By.ID,"MENU_PREPARACION")
```



Build #29 (Dec 8, 2022, 9:52:50 PM)



No changes.



Started by user [PdS-GrupoGOD](#)



Revision: d921e6a676a1a119c5401c77eb913f56098a73e1

Repository: <https://github.com/MaxGGx/SoftwareTesting-Integration-Process.git>

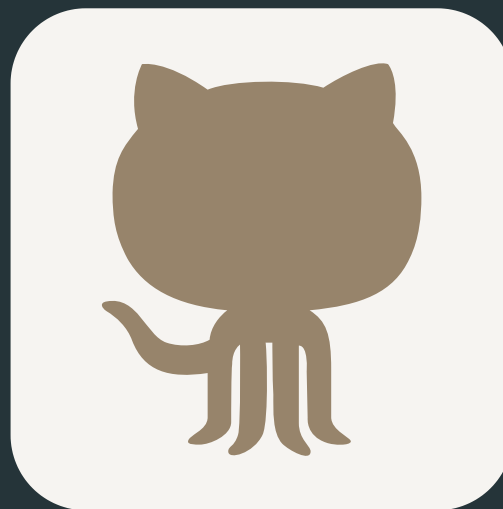
- refs/remotes/origin/main



[Test Result](#) (no failures)

GRACIAS POR VER

*Ahora pasamos a una demo de la ejecución de
las pruebas*



Acceso al GitHub (click imagen)

